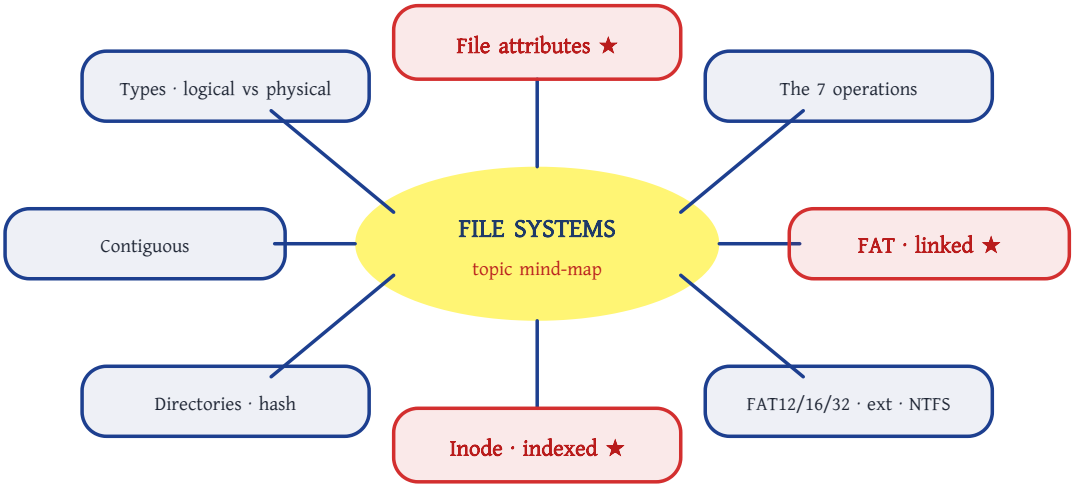
File Systems & OS Case Study

attributes · FAT · inode · directories · free space · journaling · UNIX, NT, Linux



↑ the whole topic on one page: red = high-yield, blue = supporting detail

Syllabus coverage

Operating System: "... **File Systems: files, directories, security, protection**; **Case study of some operating systems.**" Processes, scheduling, memory, I/O and deadlock are the other lessons of this chapter.

High-yield list

① 8 attributes; 7 operations
② contiguous = base + n, external frag
③ FAT = linked, links in a table
④ inode: 10 direct + 1, 2, 3 indirect
⑤ FAT12 2^{12} · FAT16 2^{16} · FAT32 2^{28}
⑥ journal = write-ahead log, metadata
⑦ NT → registry; Linux → modules

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

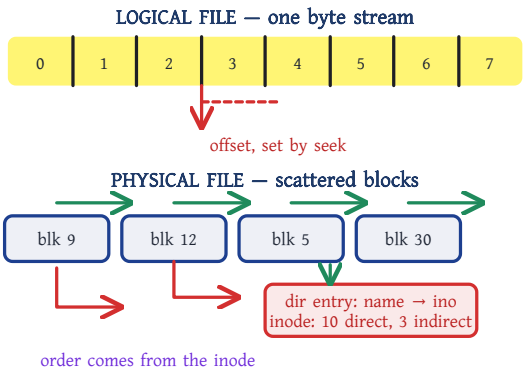
Contents

✓ Files & attributes ... p. 2
✓ The 7 operations & access ... p. 3
✓ Contiguous & linked (FAT) ... p. 4
✓ Indexed & the inode ... p. 5
✓ Directory structures ... p. 6
✓ Trace: open → read → close ... p. 7
✓ Free space & protection ... p. 8
✓ FAT/NTFS/ext/XFS, journal ... p. 9
✓ Case study: DOS, UNIX, NT, Linux ... p. 10
✓ Night-before sheet ... p. 11
✓ Practice MCQs + written ... p. 12

1 The File System and the File

Definition: A **file system** is how the OS **names, stores, shares and protects** data on secondary storage, giving programs one **device-independent** view — a tree of named files, not raw blocks. Two halves: **physical** (which blocks belong to a file) and **logical** (what a program may do with it). The disk is already cut into **blocks of 512 B – 4 KiB**.

Attribute	What it records	Exam hooks
1 name	the symbolic name	directory entry, not the inode
2 identifier	the number the OS uses	inode no · first cluster
3 type	text, binary, exec, special	characters vs packed records
4 location	pointer(s) to data blocks	start block, chain or index
5 size	length in bytes and blocks	fixed field ⇒ alloc size
6 protection	who may read / write / exec	mode bits · ACL · ACE
7 timestamps	creation, modification, access	UNIX: atime, mtime, ctime
8 owner	user id and group id	only root may change it



the program sees 0, 1, 2, 3 ...; the disk holds 9, 12, 5, 30

Logical file = the bytes the program sees, from offset 0. **Physical file** = the blocks on the medium + the directory entry / inode pointing at them. The **logical → physical mapping** is the **file organisation**.

Types. text (.txt .md .csv) · **binary** (.jpg .mp4 .exe) · **executable** (machine code; ELF on UNIX) · **special** (UNIX devices, FIFOs, sockets: /dev/sda, /dev/null).

Nothing inside a file marks where a "record" ends — **the format does**, so one file can be read byte-wise, record-wise or block-wise.

Memory trick: "The inode holds everything **except the name.**" Attribute 1 sits in the directory, 2–8 in the inode. That split explains hard links, *ln -s* and every path lookup.

Exam trap: A file's **identifier is not its name**, and the **superblock is not a file's metadata** — it describes the volume. A file system manages **secondary** storage; RAM holds buffers.

Worked example ✍ decode one directory line: *-rwxr-xr-- 1 rahim exam 8200 Sep 9 10:14 report.txt* → **-** regular file (d = directory) · **rwxr-xr-- = 7 5 4**: owner rahim rwx, group exam r-x, others r-- · link count 1 · size **8 200 B** = 17 blocks of 512 B.

2 The Seven Standard File Operations

Definition: A file is an **abstract data type**: a named, protected sequence of bytes. Every operating system gives it **exactly seven operations**, and every system call is one of them.

Operation	What it does	Typical call
create	make a new empty file, add an entry	creat(), open(O_CREAT)
delete	remove the entry, free the blocks	unlink(), remove()
open	resolve name, read metadata, check rights	open()
close	flush, release the descriptor	close()
read	copy bytes from the offset to a buffer	read(), fread()
write	copy bytes to the file at the offset	write(), fwrite()
seek	move the offset: SET / CUR / END	lseek(), fseek()

The byte-stream model ✍️: Everything follows from **one number**: the offset, initially **0**.

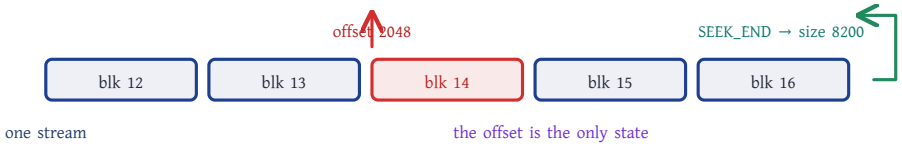
```
fd = open("/home/rahim/report.txt", O_RDONLY);
n = read(fd, buf, 512);       /* 0 → 512 */
p = lseek(fd, 0, SEEK_END);   /* p = size */
close(fd);
```

Sequential access reads from the start, record by record. **Direct** access turns block *n* into a physical block in **O(1)**.

Allocation	Block n found by
contiguous	start + n
linked / FAT	FAT[n]
indexed / inode	index entry n

Memory trick: "Create before you open, open before you read, close when done." Only read and write touch bytes.

Exam trap: **append** is not an eighth operation: it is **write** with the offset at the end. **rename** is a directory operation.

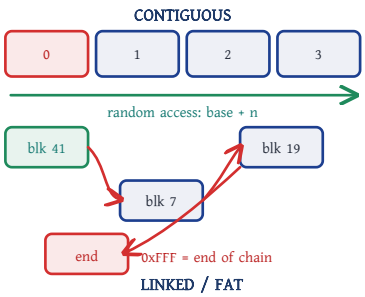


seek moves the cursor between blocks; read/write then move it again

Worked example ✍️ turn an offset into a disk address: A **contiguous** file starts at **block 12**, block size **512 B**. *lseek(fd, 2048, SEEK_SET)* → logical block 2048 ÷ 512 = **4**, offset in block 2048 mod 512 = **0**; the transfer starts at **block 16**, so reading 1 KB touches **blocks 16 and 17**.

3 File Organisation I — Contiguous & Linked

Definition: **File organisation** decides *which disk blocks* belong to a file, and so how a logical block number becomes a physical one.



contiguous: one seek, base + n · linked: walk the chain

Contiguous keeps the blocks **neighbouring**, so one seek serves the file. Linked scatters them and keeps the next address in each block, so the file grows anywhere.

Point	Contiguous	Linked
Directory holds	start block + length	first block only
Random access	**O(1)**: base + n	O(n): walk the chain
Grows later	must be **moved**	free, anywhere
Fragmentation	**external**	none outside
Cost, damage	no pointer; loses 1 block	1 pointer; truncates file

4 The FAT — Linked Allocation with a Table

Definition: The **File Allocation Table (FAT)** is linked allocation in which the "next block" pointer of **every block on the volume** is kept in **one table at the start of the volume** instead of inside the data blocks. Entry *i* holds the block after block *i* of the same file; **0 = free**, **0xFFF = end of chain**.

Memory trick: "A **F**AT is **linked** allocation whose links were moved into a **t**able."

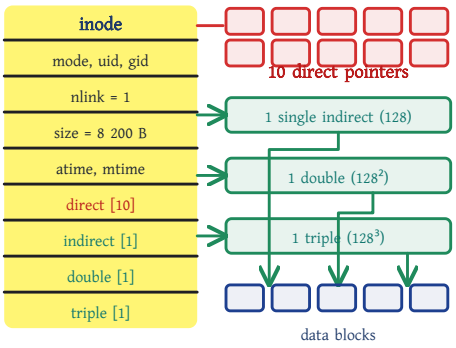
Exam trap: A FAT is **not** contiguous and **not** indexed. It is **per volume, not per file**.

Worked example ✍️ what does the FAT cost?: A 1.44 MB floppy has 2 880 sectors of 512 B: **1** boot sector + **2 × 9** FAT12 sectors + **14** root-directory sectors, leaving 2 880 − 1 − 18 − 14 = **2 847 data clusters**. Check: 2 847 × 12 bits = 4 271 B → **9 sectors** ✓.

Worked example ✍️ the allocation-size problem: A **4 096 B** file with 512-B blocks needs 8 blocks, so contiguous allocation must **declare a length** at create time. With a 16-bit length field a file holds at most 65 535 blocks = **32 MiB**; appending past it forces the file to be **relocated**. The fix is **extents** — a list of (start, length) runs.

5 Indexed Allocation & the UNIX Inode

Definition: **Indexed allocation** gives each file an **index block** — one block whose entries are the addresses of the file's data blocks. To reach logical block *n* the OS reads the index block and takes entry *n*: **no external fragmentation**, direct access, and the file may grow anywhere. Its one weakness is **internal fragmentation** — a 3-block file still burns a whole index block. **Multi-level indexing** lets index blocks point to index blocks, so the reach grows as n^k instead of n.



the inode's 13 addresses reach ≈ 1 GB with 512-B blocks

Level	Reach	Reads
direct (10)	blocks 0–9	1
single indirect	+ 128 blocks	2
double indirect	+ 16 384	3
triple indirect	+ 2 097 152	4
total	**2 113 674**	**≈ 1.01 GiB**

Where **128** = block size ÷ pointer size = 512 B ÷ 4 B. Modern **ext2/ext4** keeps the shape with **12 direct + 1 + 1 + 1 = 15** pointers; with 4 KiB blocks and 1 KiB indirect blocks (1 024 addresses) it reaches 4 MiB + 4 GiB + 4 TiB.

An **i-node** ("index node") holds the metadata **and** the pointers. A **hard link** is a second directory entry on the same inode, so the file dies only when **nlink** falls to 0 — and because the name is *not* in the inode, one file can have many names.

Worked example 1 ✍️ locate logical block 20: Direct covers 0–9, so block 20 is **single indirect**: 20 − 10 = **10** → the **11th** address in that block. Cost: **2 disk reads** (index, then data).

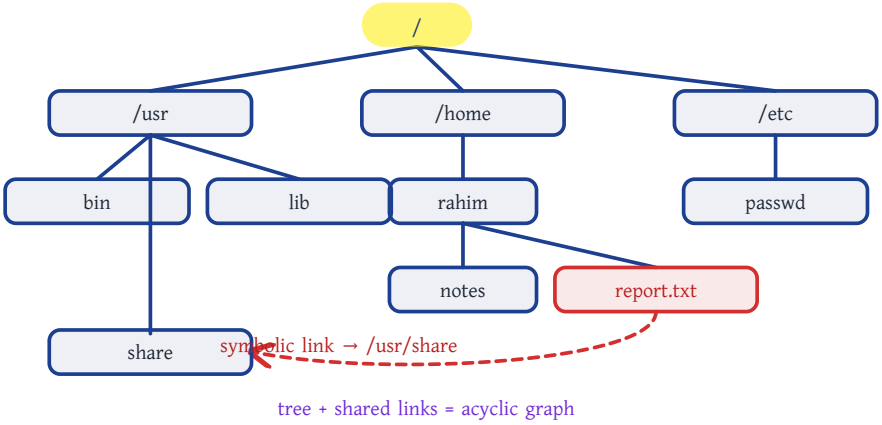
Worked example 2 ✍️ locate logical block 300: Direct + single reach only 138 blocks, so 300 is **double indirect**: 300 − 138 = 162 → sub-block 162 ÷ 128 = 1 (the 2nd) and address 162 mod 128 = **34** in it. Cost: **3 disk reads**.

Memory trick: "Subtract as you climb." Direct takes the first 10; subtract 10 and read a single indirect; if the answer is still ≥ 128, subtract 128 and read a double indirect. An indirect block is an **address book**, never data.

Exam trap: The inode stores the **name**? No — the directory does. The **superblock** holds the block bit map and the inode-table pointers, so it is volume-wide, not a file's metadata. A **file descriptor** is an integer index, not the inode.

6 Directory Structures

Definition: A **directory** is a special file whose entries map a **name to an identifier** (an inode number or the first cluster) of a file or sub-directory. It does four jobs: **grouping, naming, searching, sharing**. A directory's own "write" bit is what allows entries to be added or removed inside it.



a tree with one shared node — the acyclic graph used by UNIX-style systems

Structure	Shape	Buys	Weakness
single-level	one flat list	simplest	name clashes, no grouping
two-level	master + one per user	per-user isolation	no per-project grouping
tree	acyclic, any depth	natural hierarchy	sharing needs copies or links
acyclic graph	tree + shared dir	one copy, many parents	can dangle; ownership unclear
general graph	cycles allowed	maximum flexibility	traversal may loop
hash	h(name) → bucket	O(1) lookup	no order; poor when it grows

Path names: absolute */usr/bin/gcc*, relative *../docs/r.txt*.

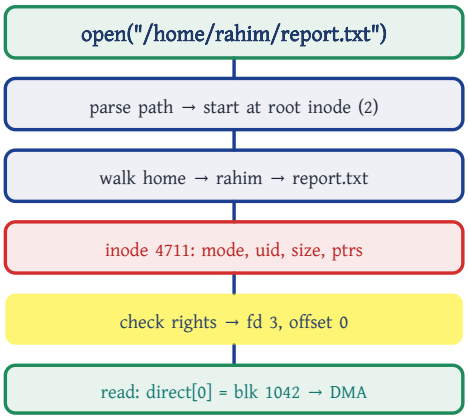
Memory trick: single → two → tree → acyclic → general → hash.

Exam trap: A symlink stores a **path**, a hard link an **inode number**.

Worked example ✍️ what does a path cost?: Opening */a/b/c/file*: read the root's block → find **a**; read a's block → **b**; read b's block → **c**; read c's block → **file**; then the inode. With **512-B blocks** and 16-byte UNIX directory entries (2-byte inode number + 14-character name) a block holds **32 entries**, so it is **4 directory reads + 1 inode read** before a single data byte moves.

7 Worked Trace: Opening a File

Definition: **open()** does not read the file — it resolves a **path name to an inode**, checks the caller's rights and returns a **file descriptor**. Data blocks are touched by the first read or write.



every component is resolved before any data is read

1. **Parse the path.** It starts with **/**, so the walk begins at the **root** (inode 2).
2. **Walk the components.** Match **home** in the root's blocks, take its inode (30); repeat for **rahim** (41) and **report.txt** → inode **4711**.
3. **Fetch the inode.** Using the inode-table pointers in the **superblock**: mode, ids, nlink, size, pointers.
4. **Check protection.** Compare the mode's owner / group / other bits with the caller's ids; a missing right fails with **EACCES**.
5. **Build the open-file entry.** Store the inode pointer, **O_RDONLY**, **offset 0**, a refcount; return the **descriptor** (**fd = 3**).
6. **read(fd, buf, 512).** Offset 0 → block 0 → the first direct pointer → **block 1042** → DMA.
7. **close(fd).** Flush, free the descriptor, decrement the refcount; at zero the inode goes.

Call	Reads from disk	Changes	Returns
open("/home/rahim/report.txt", O_RDONLY)	3 directory blocks + inode	one open-file entry, offset 0	fd = 3
read(3, buf, 512)	data block 1042	offset 0 → 512	512
lseek(3, 0, SEEK_END)	nothing	offset ← size	8200
close(3)	nothing	descriptor freed, refcount -1	0

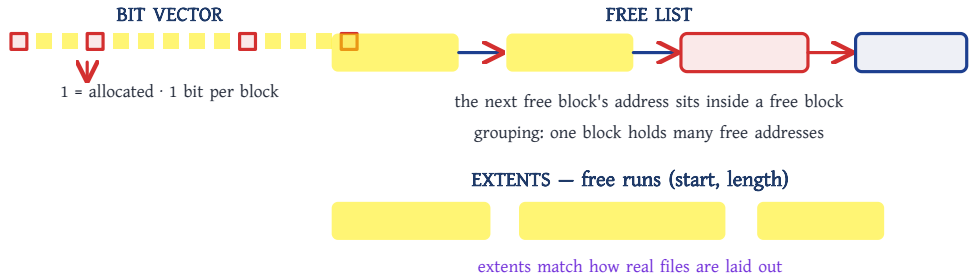
Exam trap: **open()** touches **no data block** — only directories and the inode. Every component must resolve: a missing name anywhere aborts the **whole** call.

Memory trick: "Resolve, record, return — then read": path → inode → open-file table → descriptor → data block.

IT link: This is exactly what a shell does for *./app --config conf.ini*: every component becomes a directory walk, every open an inode lookup, and the descriptor an entry in the process table that */proc/<pid>/fd* exposes. A slow NFS mount makes step 2 one network round trip per component.

8 Free-Space Management

Definition: The file system must always answer "where is the next run of free blocks?". **Free-space management** is that bookkeeping; **protection** decides which **subjects (users, processes)** may touch which **objects (files, devices)**.



three answers to "where is the next free run?"

Method	Note
bit vector	cached, very fast
linked list	no extra space
grouping	many per block
extents	one entry per run

Worked example ✍ how big is the bit vector?:
A **1 GiB** volume with 512-B blocks has $2^{30} \div 2^{9}$ = **2 097 152 blocks**, so the map needs 2 097 152 ÷ 8 = **256 KiB** — it fits in cache. A pointer list would need 4 B each = **8 MiB**.

9 Protection: Matrix, ACLs, Capabilities

Definition: The **access control matrix** has subjects as rows and objects as columns; a cell holds the rights allowed on that pair. The classical rights are **read, write, execute, append, delete, change ownership**. It can never be stored densely, so two sparse forms are used.

Form	Stored where	Catch
Access list (ACL)	per *file*: (user or group, rights)	every user on every file
Capability list	per *process*: unforgeable tokens	revoked by hand
UNIX mode	the inode: 9 bits, owner / group / others	rwx = 4 + 2 + 1
NTFS ACE	the MFT record: allow / deny per SID	deny is evaluated first

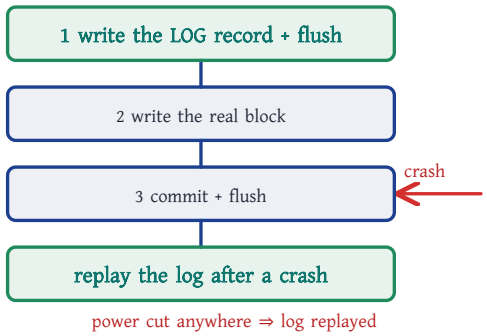
Worked example ✍ modes: *chmod 640 notes.txt* → owner **rw-** (6), group **r--** (4), others **---** (0). Special bits: setuid **4000**, setgid **2000**, sticky **1000** — so */tmp* is 1777.

Exam trap: The **matrix is the model**; the ACL is per **object** and the capability per **subject**. Protection is about **rights**; authentication and encryption are **security**.

10 File-System Types & Journaling

Point	FAT12	FAT16	FAT32
Bits per entry	**12**	**16**	**32** (28 = cluster no.)
Max clusters	2^{12} = **4 096**	2^{16} = **65 524**	2^{28} ≈ **268 M**
Cluster size	512 B	512 B–32 KB	512 B–32 KB
Max volume	≈ **2 MB**	≈ **32 MB** (2 GB)	128 GiB … **2 TB**
Names	8.3	8.3	8.3 + long (VFAT)
Where used	1.44 MB floppies	1980s–90s disks	Windows 95 OSR2

System	Journal	Defining features
FAT12 / FAT16	no	table of links, 8.3
ext2 (1992)	no	15-pointer inode, groups
ext3 (2001)	**yes**	ext2 + a metadata journal
ext4 (2006)	yes	extents, 64-bit, checksums
NTFS (1993)	yes	MFT, ACLs, streams, EFS
XFS	metadata	64-bit, allocation groups



write-ahead logging: the intent is durable before the change

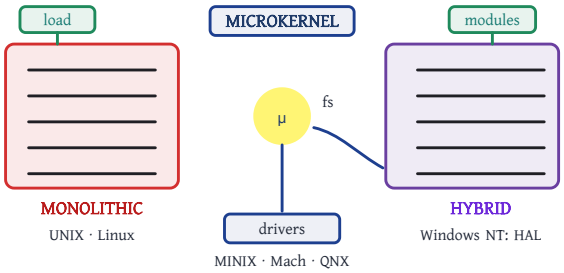
Journaling = what exactly?: A **write-ahead log** of intended **metadata** changes, flushed to stable storage **before** the change itself; on the next mount it is **replayed** and discarded. A crash therefore leaves the file system **consistent**, not the data intact: it protects the **metadata**, not the user's bytes.

Recovery without a journal: **fsck** (Linux: **e2fsck**) and **chkdsk** run at boot: verify the superblock, the bit maps, the inode table and the free-block count, rebuild the free-space map, then report or remove unreferenced inodes. It is O(volume size) — the cost a journal removes.

Memory trick: "ext3 = ext2 + a **log** (2001); ext4 = extents + 64-bit + checksums (2006)." **NTFS 1993**; **XFS** = allocation groups.

Exam trap: FAT12's limit is **4 096 clusters** (≈ 2 MB); 65 524 belongs to FAT16. A journal protects **metadata**, not data.

11 Case Study of Operating Systems



three kernel shapes; Linux is monolithic but loadable

MS-DOS (1981): Single-user, **single-tasking**, real mode on the 8086, **640 KB** conventional memory, **no memory protection**. FAT12 / FAT16, 8.3 names.

UNIX (Bell Labs, 1969–70): Ken Thompson and Dennis Ritchie; Version 1 in **1970**, renamed "Unix" in 1972, rewritten in **C** in 1973. Multi-user, multi-tasking; **everything is a file**; **pipes** join commands.

Windows NT (1993): A **hybrid kernel** — microkernel core, large executive, user-mode subsystems — plus a **HAL** hiding the hardware. Its signature is the **registry**, a settings database that replaced the DOS text files.

Linux (announced 14 August 1991): Linus Torvalds, inspired by MINIX and UNIX. A **monolithic kernel**, modular at run time through **loadable kernel modules**. **Everything is a file**; the **VFS** makes ext4 and XFS look identical.

Point	MS-DOS	UNIX	Windows NT	Linux
Origin	IBM / MS, 1981	Bell Labs, 1969	Microsoft, 1993	Torvalds, 1991
Kernel	none protected	monolithic	**hybrid**	mono + **modules**
Users / tasks	1 / 1	many / many	many / many	many / many
Config	CONFIG.SYS	text /etc	**registry**	text /etc
File system	FAT12 / FAT16	inodes, ext4	**NTFS**, FAT	ext4, XFS
All files?	no	**yes**	no	**yes**
Protection	none	rwx, groups	ACLs, SIDs	rwx, groups

Exam trap: **Windows 1.0 (1985)** was a **GUI shell on top of DOS**, not a multitasking OS — that line is **Windows NT (1993)**.

Memory trick: "UNIX and Linux are **files**, Windows is a **database**." Shapes: monolithic · microkernel · hybrid.

Night-Before-the-Exam Sheet

Fact sheet:

block size 512 B – 4 KiB

8 attributes, 7 operations

contiguous: base + n, external frag

linked: walk the chain

FAT = linked, links in a table

FAT entry 0 free, 0xFFF end

FAT12 2^{12} = 4 096 clusters ≈ 2 MB

FAT16 2^{16} = 65 524 ≈ 32 MB

FAT32 28-bit cluster no. ≈ 2 TB

floppy: 2 880 sectors → 2 847 data

inode: 10 direct + 1, 1, 1 indirect

128 addrs per 512-B indirect

10+128+16 384+2 097 152 blocks

= 2 113 674 ≈ 1.01 GiB

ext2/4: 12 direct + 1 + 1 + 1

UNIX dirent 16 B → 32 per block

symlink = path, hard link = inode

1 GiB/512-B: 2 097 152 → 256 KiB

free: bit vector, list, grouping, extents

ACL per object, capability per subject

rwx = 4+2+1; 4000/2000/1000

ext2 1992 · ext3 2001 · ext4 2006

NTFS 1993: MFT, ACLs, $LogFile

journal = write-ahead log, metadata

fsck / e2fsck / chkdsk

DOS 1981 · UNIX 1969 · NT 1993

Linux 1991; NT kernel = hybrid

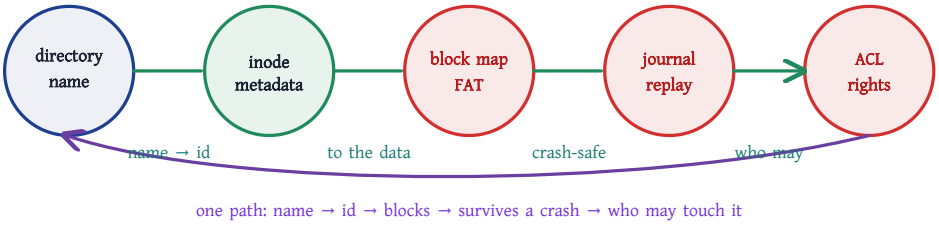
UNIX & Linux: everything is a file

Top 12 one-liners for MCQs

- A file's **name** is in the directory, not the inode
- Standard file operations = **7**
- Contiguous random access = base + n
- Contiguous suffers **external** fragmentation
- FAT = linked allocation with a table
- FAT12 max = **4 096** clusters
- ext3 = ext2 + a journal
- A journal protects **metadata**, not data
- ACL per object, capability per subject
- Hard link = inode number, symlink = path
- Windows NT replaced DOS text files with the **registry**
- UNIX and Linux: **everything is a file**

Top traps: FAT12 = 4 096 clusters, not 65 524 · FAT is linked, not contiguous or indexed · the inode has no file name · 512 ÷ 4 = 128 addresses per indirect block · a bit vector needs n/8 bytes · a journal protects metadata · Windows 1.0 (1985) was a GUI on DOS.

All tricks together: The inode holds everything except the name · FAT = linked links hoisted into a table · Contiguous = base + n, so it is fast · Subtract as you climb the indirect levels · a symlink stores a path, a hard link an inode number · ext3 = ext2 + a log · UNIX is files, Windows is a database.



the whole file system in five nouns

✓ Practice MCQs (exam pattern, 10 marks)

1. How many standard file operations are there?

(a) 5 (b) 7
(c) 9 (d) 4

2. In UNIX/Linux a file's metadata and block pointers live in:

(a) the superblock (b) the boot block
(c) the inode (d) the file descriptor

3. The FAT is a variation of which allocation method?

(a) linked allocation (b) contiguous allocation
(c) indexed allocation (d) multi-level indexed allocation

4. A 512-B block of 16-byte UNIX directory entries holds:

(a) 16 entries (b) 64 entries
(c) 8 entries (d) 32 entries

5. ext3 differs from ext2 by adding:

(a) an inode table (b) a journal (log)
(c) clusters (d) a FAT

6. How many clusters can FAT12 address at most?

(a) 4 096 (b) 65 524
(c) 2^{28} (d) 2^{16}

7. Free space recorded as (start, length) pairs is called:

(a) a bit vector (b) a linked list
(c) extents (d) an inode

8. A 1 GiB volume with 512-B blocks needs a bit vector of:

(a) 64 KiB (b) 128 KiB
(c) 512 KiB (d) 256 KiB

9. Windows NT replaced the DOS text configuration files with:

(a) CONFIG.SYS (b) the registry
(c) the MFT (d) a batch file

10. A contiguous file starts at block 12; its 5th block is:

(a) block 5 (b) block 12
(c) block 16 (d) block 17

Answer key → 1-b · 2-c · 3-a · 4-d · 5-b · 6-a · 7-c · 8-d · 9-b · 10-c

Written question: A UNIX file system uses 512-byte blocks, 4-byte pointers and the classic inode: 10 direct addresses plus one each of single, double and triple indirect. (i) Largest file? (ii) Locate logical blocks 20 and 300? (iii) Two pros and two cons of a FAT. 10

1. **Per indirect block** — 512 ÷ 4 = **128.** 1
2. **Max file** — 10 + 128 + 16 384 + 2 097 152 = **2 113 674 ≈ 1.01 GiB.** 2
3. **Block 20** — direct covers 0–9 ⇒ **single indirect**, index 10; **2 reads.** 2
4. **Block 300** — direct + single reach 138 ⇒ **double indirect**, 162, sub-block 1, address 34; **3 reads.** 2
5. **FAT — pro** walk the chain without reading data. **Con** per-volume table grows with the disk. 3

Before the exam ✓

- 8 attributes · 7 operations
- inode arithmetic · open → close

– contiguous / linked / indexed

– FAT limits · journal · four OS